1. Dharshini

Introduction and project overview

FOUR-PERSON PRESENTATION SCRIPT

Read the normal text aloud. ACTION boxes are instructions for the demonstration; do not read them aloud. Each speaker starts on a new page.

Good morning, ma'am. We are Dharshini, Kani, Hemanath, and Ibadul Azam.

Our project is "Zero-Trust Enterprise Storage Vault with Automated Ransomware Defense and Disaster Rollback."

The main idea is to protect important files, control who can access them, detect a suspicious change, and recover the original files when needed.

We developed this using Bash shell scripting and Linux commands. The application runs through a menu in the Ubuntu terminal.

To explain the problem, let us take a simple example.

A student stores an assignment on a shared computer. The student should be able to read and update that assignment, but an outsider should not be allowed to access it.

We also need a protected backup. If the working files are damaged, we should have a clean copy available for recovery.

Our project brings these requirements together using Linux permissions, file-integrity checking, automatic monitoring, and backup recovery.

We use three sample files.

Dharshini - continued

Assignment.txt contains the student's coursework. Marks.txt contains fictional marks. Canary.txt is a security trap file that should not change during normal work.

We also have separate accounts for an authorized student and an outsider. The administrator manages the application and performs recovery.

In our project, the term zero trust refers to giving limited access and repeatedly checking the canary's integrity.

This is an educational prototype. It demonstrates selected security principles rather than a complete enterprise security system.

ACTION: Open Ubuntu and run: sudo ztvault

This is our terminal interface.

The menu acts as the frontend. It shows the available operations and accepts the user's choices.

The Bash script handles the backend processing. It reads the choice, calls the corresponding function, executes Linux commands, and displays the result.

I will now hand over to Kani to explain access permissions and normal file operations.

2. Kani

Access control, backup, and user interaction

Thank you, Dharshini.

I will explain how we control access to the files.

We created two Linux test accounts. Ztv_writer represents the authorized student, and ztv_outsider represents someone who should not have access.

The administrator starts the menu using sudo. However, student operations are performed through the student account using the runuser command.

This is important because we need to test what the student can actually do, rather than performing every action as the administrator.

ACTION: Choose option 1.

Here we can see the assignment and the sample marks.

These files are stored inside our vault directory, which is /srv/ztvault/data.

ACTION: Choose option 3.

Now I am checking the access permissions.

The authorized student can read the assignment.

When the outsider tries to read it, Linux displays "Permission denied."

The student is also denied access to the backup folder.

These permission errors are expected. The tests show PASS because the restricted actions were successfully blocked.

During normal operation, the vault allows access to the administrator and the authorized student group. Other users have no access.

Kani - continued

The backup and trusted fingerprints are stored separately in administrator-only directories.

ACTION: Choose option 4.

This option creates a clean backup if one does not already exist. If the backup is already present, it verifies the existing copy.

In our demonstration, the clean backup already exists, so the application verifies it and preserves it.

We use tar to create the backup archive and SHA-256 checksums to verify its integrity.

We do not replace the original backup every time the assignment changes, because we need that clean version for recovery.

ACTION: Choose option 2 and enter: I completed my Linux assignment.

Now I am entering a sentence.

The script checks that the input is not blank and does not exceed 200 characters. It then adds the sentence to the assignment using the student account.

ACTION: Choose option 1.

We can see the sentence in the assignment, which confirms that the input was saved.

ACTION: Choose option 5.

The security check still reports SAFE.

This is correct because editing the assignment is an allowed operation. We have not changed the canary file.

Hemanath will now explain how the canary helps us detect a suspicious change.

3. Hemanath

Detection, monitoring, and access blocking

Thank you, Kani.

The canary is a file that normal student work should not change.

We can think of it like a security seal. If the seal changes, it tells us that something needs attention.

Our project saves the original SHA-256 fingerprint of this file. A fingerprint is a value calculated from the file's contents.

During a security check, the script compares the current canary with the saved fingerprint.

If they match, the result is SAFE. If they do not match, the script records an incident and restricts student access.

SHA-256 is used here to detect changes. It does not encrypt the files.

ACTION: Choose option 10.

I am now enabling automatic monitoring.

We use a systemd timer to run the security check approximately every ten seconds while Ubuntu is running.

The timer starts a service, and the service runs our Bash script in monitoring mode.

This allows the checks to happen without someone repeatedly choosing the manual-check option.

ACTION: Choose option 6 and type DEMO.

Now I am simulating a suspicious change.

This option safely changes our sample canary file. We are not running actual ransomware or encrypting any files.

Hemanath - continued

We are creating the condition that our monitor is designed to detect: a canary that no longer matches its original fingerprint.

ACTION: Return to the menu and wait about 15 seconds.

The monitor checks the canary and detects the mismatch.

It then changes the vault's directory permissions so that student access is blocked. It also creates an incident marker and records an alert.

ACTION: Choose option 7.

Here we are trying to write to the vault as the student.

Linux denies the write, and our test confirms that access is blocked.

Before the incident, the student could update the assignment. After detection, this write is denied.

This is the containment step in our project.

ACTION: Choose option 9.

This is the security log.

It records events with timestamps, including the simulated canary change and the alert.

These records help the administrator understand what happened and when it happened.

One limitation is that our detection depends on the canary. If an attack changes other files without touching the canary, this check may not detect it.

Also, changing directory permissions restricts new student access. It does not stop root or revoke access through a file that a process already has open.

I will now hand over to Ibadul to demonstrate recovery and error handling.

4. Ibadul Azam

Recovery, error handling, code, and conclusion

Thank you, Hemanath.

We have detected the suspicious change and restricted access. Now we need to restore the original files.

Detection and access blocking can happen automatically, but recovery requires the administrator's confirmation.

ACTION: Choose option 8.

The application asks us to type RESTORE before continuing.

This confirms that we want to replace the current working files with the clean backup.

ACTION: Type RESTORE.

The script verifies the backup and checks that the recovered files match their saved original fingerprints.

It also preserves the previous vault directory.

After successful recovery and verification, it restores normal access to the vault.

ACTION: Choose option 1.

We can now see the original assignment again.

The sentence that Kani added earlier is no longer present. That is expected because recovery returns the files to the state saved in the clean backup.

Changes made after that backup are not included in the restored version.

ACTION: Choose option 5.

The security check reports SAFE again because the original canary has been restored.

Ibadul Azam - continued

ACTION: Enter 99 as the menu choice.

Next, I am demonstrating error handling.

Ninety-nine is not a valid menu choice. The application displays an error and allows the user to choose again.

ACTION: Choose option 2, then press Enter without typing a sentence.

Here I am submitting empty input.

The script rejects it instead of saving an empty entry. We also check the input length and require the correct confirmation words for simulation and recovery.

I will briefly explain how the code is organized.

We use variables to store paths and account names. Functions separate the main operations, such as updating the assignment, checking security, and recovering files.

A while loop keeps the menu running. The read command accepts input, and a case statement selects the correct function.

If conditions are used to validate input and check whether an operation can proceed.

Some important commands are chmod for permissions, runuser for testing different accounts, sha256sum for fingerprints, tar for backups, and systemctl for monitoring.

We also use flock to prevent operations from overlapping. For example, monitoring should not check the vault halfway through recovery, so the operations take turns using a shared lock.

Our current backup is local and contains one clean version. Future improvements could include backups on another device, multiple backup versions, and broader file monitoring.

Through this project, we demonstrated user management, permissions, Bash scripting, user interaction, automatic monitoring, logging, backup verification, and recovery.

Our demonstration showed that the authorized student could update the assignment, the outsider was blocked, a canary change triggered automatic access restrictions, and the administrator could restore the original files.

Thank you, ma'am. We are ready to answer your questions.

Demonstration quick reference

Keep this page beside the laptop. Return to the menu after each operation.

Speaker	Menu sequence
Dharshini	Start: sudo ztvault
Kani	1 > 3 > 4 > 2 > 1 > 5
Hemanath	10 > 6 > type DEMO > wait about 15 seconds > 7 > 9
Ibadul Azam	8 > type RESTORE > 1 > 5 > 99 > 2 > empty input

Before presenting

If the vault is still locked from an earlier test, choose option 8 and type RESTORE to prepare it for the demonstration.

Important distinction

Detection and access blocking can happen automatically. Recovery requires the administrator to choose it and type RESTORE.

Timing

Allow roughly 12-15 minutes with the live demonstration. Rehearse once and adjust your pace to the time given by your faculty.